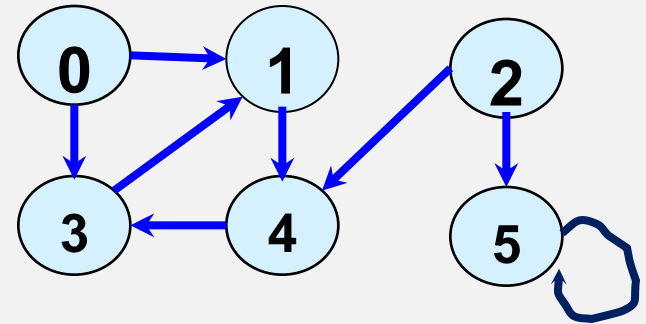
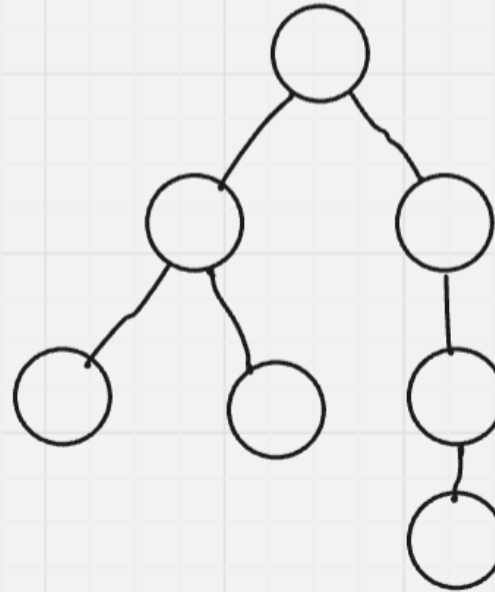
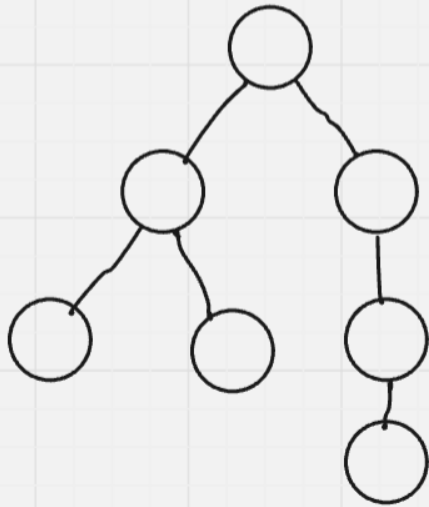




graph traversal

- กระบวนการเข้าไปเยือนโหนดในกราฟ
- แต่ละโหนดจะถูกเยือนเพียงครั้งเดียว
- ในกราฟระหว่างโหนดอาจจะมีหลายเส้นทาง ดังนั้นเพื่อป้องกันการท่องไปในเส้นทางที่ซ้ำเดิมจึงจำเป็นต้องทำเครื่องหมายบริเวณที่ได้เยือนเสร็จเรียบร้อยแล้วเพื่อไม่ให้เข้าไปเยือนอีก
- วิธีการท่องกราฟมี 2 แบบดังนี้
 - BFS
 - DFS



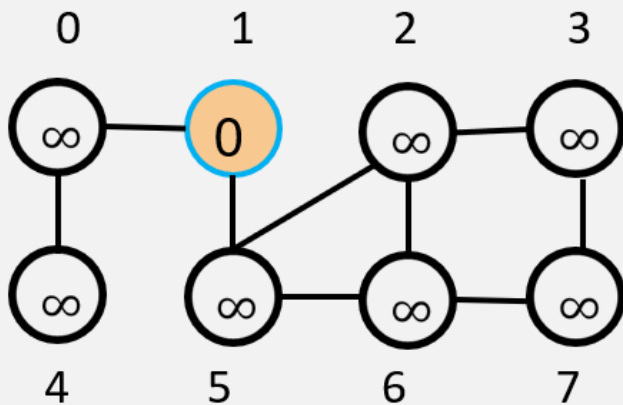


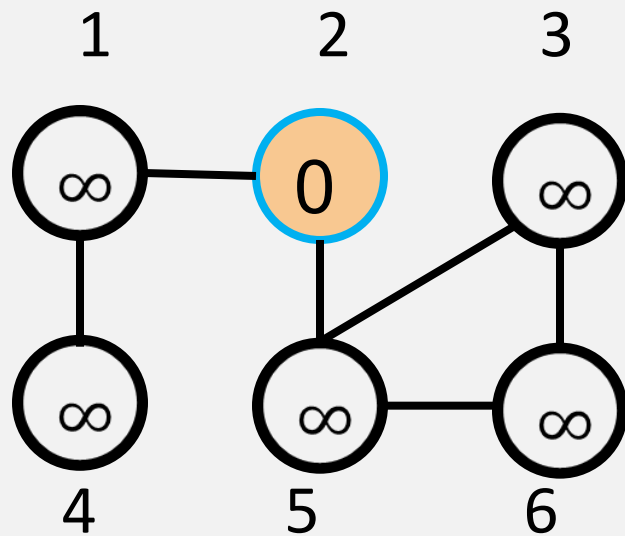


4.3 Breadth-first search (BFS) : Algorithm for searching a graph

Definition : Given a graph $G = (V, E)$ and distinguished source vertex s , bfs systematically explores the edges of G to discover every vertex that is reachable from s to all such reachable vertices.

BFS จะสำรวจ **เส้นเชื่อม (edges)** ของกราฟ G อย่างเป็นระบบ เพื่อค้นหา **จุดยอด (vertices)** ทุกจุดที่สามารถเข้าถึงได้จากจุดเริ่มต้น s และสำรวจไปยังจุดยอดที่สามารถเข้าถึงได้เหล่านั้นทั้งหมด





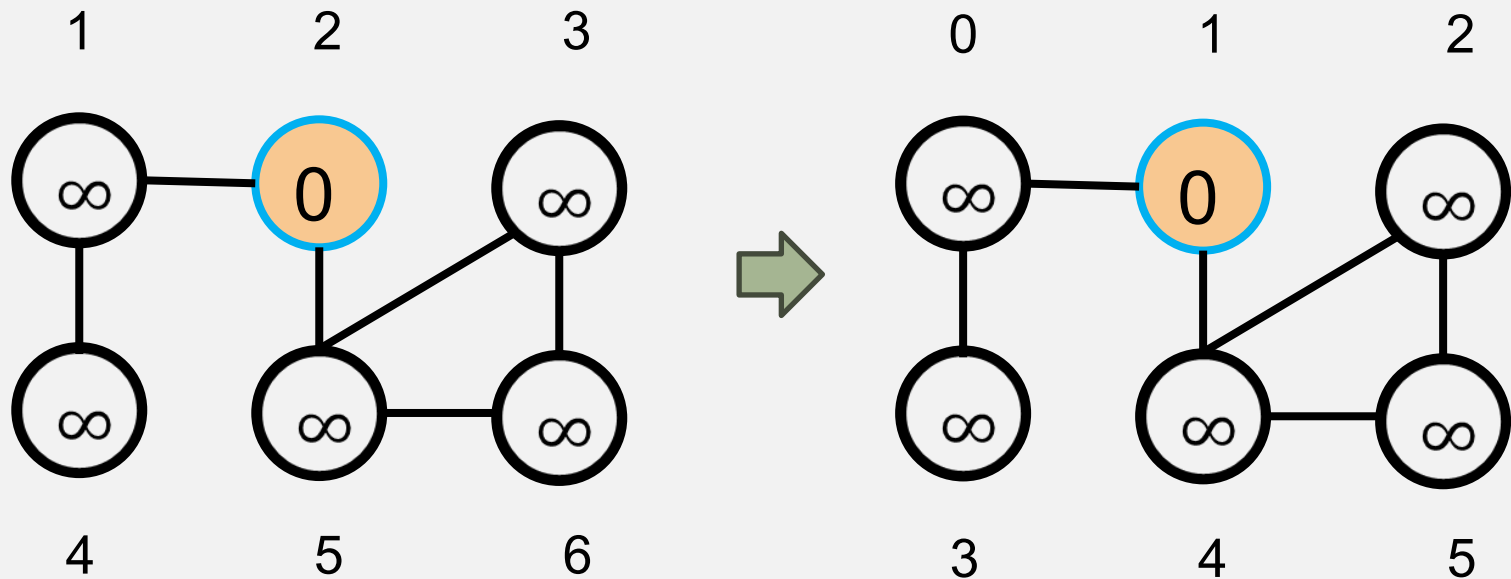
Q 2

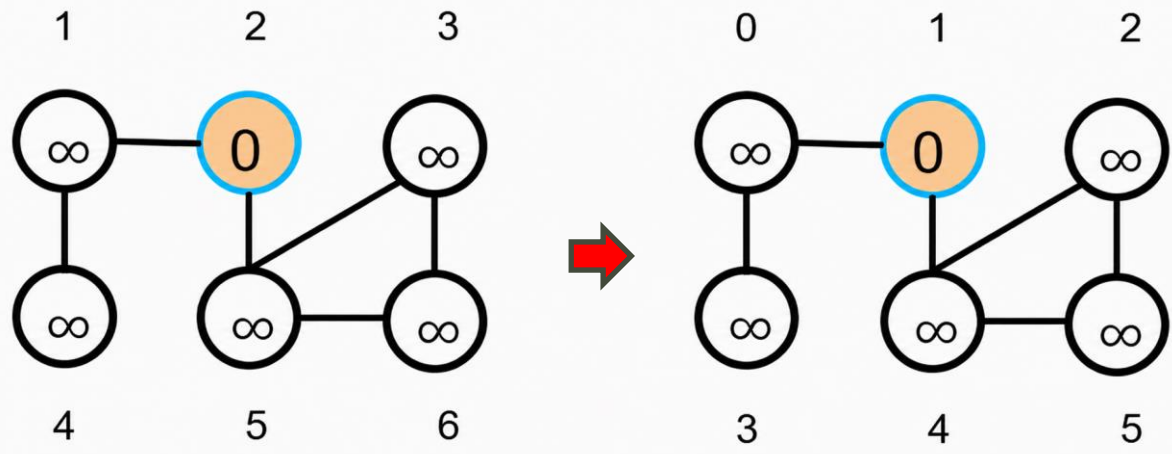


ตัวอย่าง

Output

- Compute distance
- Produces a bfs tree.





			visited	layer	pred
0		1 → 4			
1		0 → 4			
2		4 → 5			
3		0			
4		1 → 2 → 6			
5		4 → 2			



4.3.1 BFS Algorithm

BFS(G, s)

for each vertex $u \in V[G] - \{s\}$

do $\text{visited}[u] = F$

$\text{layer}[u] = \text{INFINITE}$

$\text{pred}[u] = -1$

$\text{visited}[s] = T$

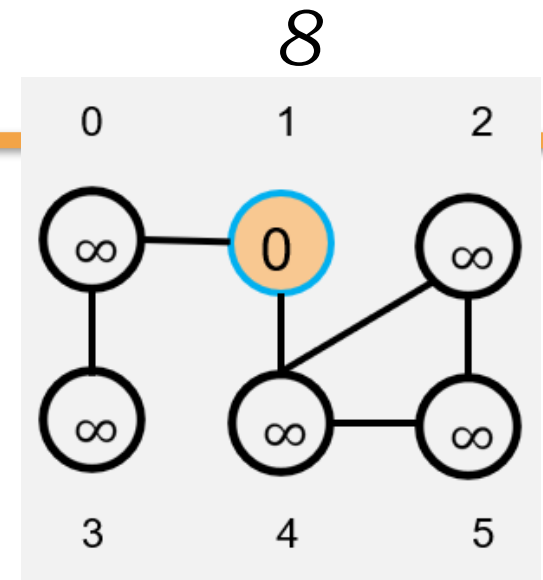
$\text{layer}[s] = 0$

$\text{pred}[s] = s$



03603212 : Module4 – Graph

```
Enqueue(s);
while (queuesize() != 0)
{
    u = head(q)
    for each neighbor v of u
    {
        if (visited[v]=F)
        {
            visited[v] = T
            layer[v] = layer[u]+1
            pred[v] = u
            enqueue(v)
        }
    }
    dequeue()
}
```



			deg	visited	layer	pred
0		1 → 4	0			
1		0 → 4	1			
2		4 → 5	2			
3		0	3			
4		1 → 2 → 6	4			
5		4 → 2	5			



```
#include <iostream>
#include <vector>
#include <list>
using namespace std;

#define MAX_N 100000

int n, m; // vertices edges
vector<int> adj[MAX_N];
int deg[MAX_N];
bool visited[MAX_N];
int layer[MAX_N];
int pred[MAX_N];
list<int> Q;
```

```
void init()
{   for(int i=0;i<n;i++)
    {
        deg[i]=0;
        visited[i]=false;
        layer[i]=-1;
        pred[i]=-1;
    }
}

void read_input()
{
    .....
}

void show()
{
    .....
}
```



```
void bfs(int s)
```

```
{ int u=s;
```

```
  visited[s]=true;
```

```
  layer[s]=0;
```

```
  pred[s]=0;
```

```
  Q.push_back(s);
```

```
while(!Q.empty())
```

```
{ int u=Q.front();
```

```
  Q.pop_front();
```

```
  visited[u]=true;
```

```
  for(int d=0; d< deg[u] ; d++)
```

```
  { int v=adj[u][d];
```

```
    if(visited[v]==false)
```

```
    {
```

```
      layer[v]= layer[u]+1;
```

```
      Q.push_back(v);
```

```
    }
```

```
  }
```

```
}
```

```
for(int i=0;i<n;i++)
```

```
{
```

```
  cout<< i+1 << ", " << layer[i] << endl;
```

```
}
```

```
}
```

```
int main()
```

```
{ init();
```

```
  read_input();
```

```
  show();
```

```
  bfs(0);
```

```
}
```



1. Edges are explored out of the most recently discovered vertex v that still has unexplored edge leaving it.
2. When all of v 's edges have been explored, the search backtracks to explore edges leaving the vertex from which v has discovered. Until we have discovered all the vertices that are reachable from the original source vertex